QAgent: A Question-Generating Agent for Mathematical Research

From paper reading to QED-ready theorem-level problems

QAgent Project

June 4, 2026

Core Objective

What is QAgent trying to do?

Given a small collection of mathematical papers, QAgent generates theorem-level research problems that can be handed to GPT-Pro or a QED-like proving agent.

QAgent is not designed to produce vague open-problem titles. Its final output should include:

- a precise model, object class, assumptions, and conclusion;
- a problem decomposable into lemmas, estimates, and compactness steps;
- survey and duplicate-risk evidence;
- proof guidance, verification rules, feasibility analysis, and metadata;
- a realistic path toward a small publishable mathematical result.

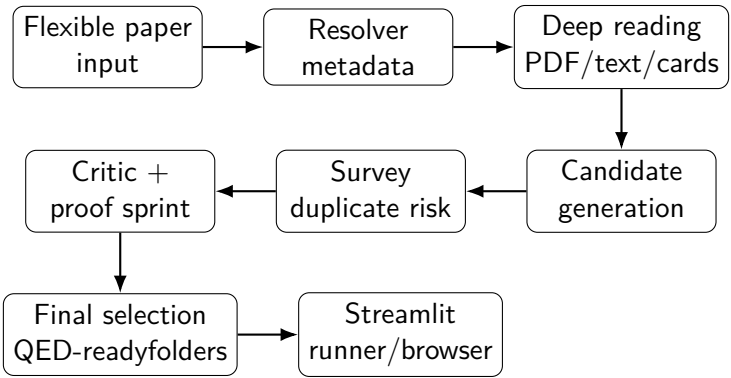
Why Not Just Ask an AI Directly?

Direct prompting from abstracts often produces low-quality questions:

- **Keyword splicing:** "regularity" becomes a generic regularity problem.
- **Theorem reproduction:** the main theorem of the input paper is repackaged as a new problem.
- **Template titles:** for example, "Sharp threshold for the main regularity mechanism".
- **Wrong-domain mechanisms:** no-neck, varifolds, Yamabe, or De Giorgi appear where they do not belong.
- **No proof route:** the problem sounds interesting but has no first lemma or plausible proof sprint.

QAgent is built around quality gates rather than one-shot generation.

System Architecture



Two Operating Modes

Deep Mode

- For 1–10 papers.
- Quality is prioritized over speed.
- Attempts metadata, PDF, and full-text reading.
- Extracts theorem, proof, method, limitation, and gap cards.
- Runs survey, critic review, and proof sprints.
- Intended for final theorem-level problem generation.

Batch Mode

- For 11–60 papers.
- Fast screening only.
- Outputs are marked lower confidence.
- Recommends which papers deserve Deep Mode.
- Not advertised as final SCI-level QED-ready generation.

Deep Reading Stage

Before asking for research questions, QAgent tries to understand the paper mechanism:

- `paper_profile.json`: title, authors, model class, objects, methods, confidence.
- `theorem_cards.json`: theorem labels, assumptions, conclusions, dimension, boundary conditions.
- `proof_cards.json`: proof strategy, key lemmas, key estimates, fragile steps.
- `method_cards.json`: reusable tools such as blow-up, monotonicity, Caccioppoli, compactness.
- `limitation_cards.json`: dimension restrictions, smoothness, smallness, missing rates.
- `gap_cards.json`: possible research gaps derived from theorem and limitation cards.

If only metadata or abstract is available, final outputs must be lower confidence.

Candidate Generation: From Templates to Mechanisms

The old risk was overfitting to transfer templates.

QAgent now uses a two-level transfer structure:

1. **Parent patterns:** perturbation, setting, dynamic, compactness/defect, quantitative, sharpness, low-regularity, proof-module.
2. **Detailed TP patterns:** subpatterns and examples, not mandatory templates.

Each strong candidate should explain:

- the source theorem or proof method;
- the target model;
- one primary new obstruction;
- why the old proof may survive;
- the smallest publishable theorem.

Domain Gate

Specialized proof mechanisms are allowed only in the right mathematical domain.

Examples

- No-neck, bubble tree, and energy identity only for concentration or bubbling problems.
- Varifold compactness, tangent cones, and sheeting only for GMT, phase-field limits, currents, or minimal surfaces.
- De Giorgi iteration only for PDE regularity problems where level-set iteration is relevant.
- Yamabe, Green functions, and positive mass only for conformal or Riemannian geometry.
- Dispersive estimates only for wave, Schrodinger, or related dispersive PDE.

Metadata now includes:

`domain_gate`, `forbidden_mechanisms_avoided`, `wrong_domain_mechanism_penalty`.

Theorem-Level Gate

The final `problem_statement.tex` must be a clean theorem-level problem:

- a specific title;
- a concrete model: PDE, energy, variational problem, geometric object, or flow;
- a concrete setting: ball, bounded domain, manifold, half-ball, varifold setting, etc.;
- a concrete object class: weak solution, minimizer, stable critical point, integral current, etc.;
- explicit assumptions: dimension, regularity, boundary conditions, energy bounds, stability, smallness;
- a concrete conclusion: estimate, compactness, regularity, uniqueness, counterexample, or sharpness.

The q environment must not contain meta text such as “Generated from metadata”, “QED suitability”, or “choose assumptions to match the paper”.

Survey and Duplicate-Risk Gate

In Deep Mode, every candidate must receive a candidate-level survey report:

- at least eight search queries;
- exact title query, model + theorem type, model + conclusion, author + related theorem;
- public metadata sources: local metadata, Crossref, OpenAlex, arXiv, Semantic Scholar;
- no Google Scholar scraping;
- classification: reproduction, proof module, likely known, transfer, new theorem-level, or too vague.

High duplicate-risk candidates cannot be final selected.

Critic Review and Proof Sprint

Each refinement round runs a proof sprint for every remaining candidate:

1. theorem-level restatement;
2. key estimate or compactness lemma;
3. 5–10 step proof attempt;
4. where the input paper method is used;
5. where the method may fail in the new setting;
6. whether failure suggests a counterexample or sharpness version;
7. QED/GPT-Pro attackability score;
8. SCI publishable potential score;
9. keep/remove reason.

If the proof route is simply "apply a known theorem directly", the candidate is removed.

Final Output Structure

Each final selected question becomes a QED-readyfolder:

- `problem_statement.tex`
- `additional_prove_human_help_global.md`
- `additional_verify_rule_global.md`
- `survey_queries.md`
- `feasibility_analysis.md`
- `metadata.json`

The main UI shows only the final selected problems, not candidate surveys or refinement logs.

Runner UI

- Paste flexible paper entries.
- Choose Deep Mode or Batch Mode.
- Set the number of papers n, refinement rounds a, and final questions b.
- Click **Run Agent**.
- The backend uses Codex CLI.
- The UI shows blue “running”, green “completed”, or red “failed” status.
- The output browser shows papers by title and questions by selected-question title.

The current system does not call the OpenAI API and does not automate the GPT-Pro web UI.

Recent Quality Improvement

We found that the active transfer catalog was too specialized. In particular, energy identity and no-neck language could contaminate unrelated papers.

Changes made:

- changed the transfer catalog from a hard template to an idea source;
- added eight parent transfer patterns;
- replaced “Energy Identity / No-Neck” with “Concentration / Defect-Control”;
- added a domain gate;
- reduced the weight of transfer-pattern fit;
- added wrong-domain mechanism penalty;
- connected the changes through runner prompt, validator, and hard review.

How We Verified the Change

We checked whether the changes actually affect the runtime pipeline:

1. **Catalog:** old TP19 title is gone; domain-gate rules are present.
2. **Question prompt:** candidates must include `domain_gate`.
3. **Runner prompt:** Codex no longer sees "must match one primary pattern".
4. **Validator:** new pattern fields enter the candidate quality audit.
5. **Hard review:** pattern fit gives only a small bonus, not a hard gate.
6. **Tests:** candidate validator, hard review, and runner prompt tests pass.

Current Limitations

- Output quality still depends heavily on full-text access and theorem-card extraction.
- Online survey uses public metadata sources and cannot replace a true literature review.
- GPT-Pro web interaction is a handoff, not automatic browser control.
- Batch Mode is screening only.
- Final theorem statements still need validator checks and human spot checks.

Next Steps

1. Improve PDF, arXiv, and CVGMT full-text fetching.
2. Make theorem-card extraction more reliable.
3. Improve source-by-source logging for online survey.
4. Add more user feedback examples of bad generated questions.
5. Run Deep Mode on 1–3 papers as a high-quality demo.
6. Compare QAgent scores against human proof-sprint judgments.

Takeaway

Main idea

QAgent first reads the paper mechanism, then generates candidates, surveys them, criticizes them, runs proof sprints, and only then selects final theorem-level problems.

The intended final problems should be:

- theorem-level and self-contained;
- not direct restatements;
- not template-driven;
- supported by a concrete new obstruction;
- plausible for QED/GPT-Pro assisted proof development;
- potentially publishable as small SCI-level results.